Morgan State Food Resource Center: A Pantry as a Research Instrument

Author: Patrick Valery **Affiliation:** Morgan State University, Spring 2026 **Date:** April 27, 2026 **Repository:** https://github.com/PadiusD1/ClaudeMSUfoodResourceCenter **Companion case study:** https://github.com/PadiusD1/morgan-frc-research-paper

Abstract

The Morgan State Food Resource Center (FRC) operates as a campus food pantry, but it also generates a stream of primary research data: which items move, who picks them up, what categories run low, what donors deliver, and where on campus pickups happen. Until this project, that data lived in paper logs, spreadsheets, and the staff's heads. This paper documents the design and implementation of a single-binary, offline-first pantry database written in TypeScript on Express and React, backed by SQLite via Drizzle ORM. The system models 18 tables and 10 enumerations across seven user-facing surfaces (dashboard, inventory, clients, donors, requests, check-in/check-out, kiosk and student portal) and treats every donation and pickup as a GPS-stamped, weighed, and itemized record. Demo data covers 41 transactions, 27 SKUs, 10 students, 6 donors, and 791.8 lbs of food moved. The deployment cost is zero recurring dollars: it runs on a single laptop or NUC, on a flash drive if needed, with no cloud dependency and no SaaS contract. The design trade-offs (single-writer SQLite, no multi-pantry federation yet, demo-seeded today) are documented honestly so future maintainers can extend the system without rewriting it.

---

1. Why I Built This

I walked into the Morgan State Food Resource Center as a student volunteer in the fall semester. Within an hour I noticed three things.

First, the pantry was busy. There was a real, recurring stream of students picking up food, and a real, recurring stream of donations coming in.

Second, the data evaporated. A donor would drop off four bags of rice. A volunteer would write "rice" on a clipboard. A week later, nobody could tell you who delivered it, what brand it was, what it weighed, what it was worth, or how much of it had been distributed.

Third, this was a research goldmine being thrown away. A campus pantry is a quasi-experimental setting. It has a defined population (eligible students), a defined supply chain (donors, drives,

purchases), measurable demand (requests, pickups, bundles), and a clean physical boundary (the room itself). Everything that happens inside it is, in principle, measurable. The only thing missing was the instrument.

So I built one. Not a charity tracker. A research instrument that happens to also run the pantry.

The hypothesis behind the build is simple: if every donation is weighed and itemized at intake, every request is captured before pickup, every pickup is GPS-stamped at check-out, and every item carries a category and a value, then the same database that operates the pantry on Tuesday afternoon can answer real research questions on Friday morning. Questions like: which categories run out first? Which donors fill which gaps? Which items are over-requested and under-stocked? What is the dollar value of unmet demand?

This paper describes how that instrument was built.

2. Design Goals and Non-Goals

The system was designed under five hard constraints and five explicit non-goals. These were chosen before a single line of code was written, and they drove every architectural decision afterwards.

Goals

1. **Zero recurring cost.** The pantry runs on volunteer labor and a small budget. A SaaS subscription is a non-starter. The system must be free to run forever, on hardware the school already owns.
2. **Offline-first.** Wifi at the FRC is unreliable. The pantry cannot stop because the internet drops. The database lives on local disk and serves over LAN.
3. **Single-writer SQLite.** One pantry, one database file, one process. No distributed-systems complexity. No replication lag. No cloud egress fees.
4. **Vendor-portable.** Schemas are defined with Drizzle ORM in a Postgres dialect, then materialized into SQLite at runtime. The same schema can move to Postgres on day one if the pantry ever federates across multiple sites.
5. **Sub-second UX.** Every interaction (search, scan, check-in, check-out) must feel instant. No spinners on hot paths. The volunteer at the desk should never wait on the database.
6. **Auditable.** Every state change on a request writes an audit-log row. Every weight and price change writes a history row. Nothing is overwritten silently.

Non-Goals

7. **Multi-tenant SaaS.** This is a single-pantry system. Federation is future work, not v1.
8. **Mobile apps.** The kiosk and the student portal run in the browser. No native iOS or Android.
9. **Predictive analytics.** The system records data; it does not (yet) forecast demand. That is future work and will require IRB approval.

10. **Demographic capture beyond what is operationally needed.** No race, ethnicity, income, or sensitive attributes. FERPA and student dignity come first.
11. **HIPAA / SOC 2 compliance.** This is a campus pantry, not a health record. We use FERPA-aware practices but do not pursue formal certification.

3. The 5W1H

Question	Answer
Who built it	Patrick Valery, undergraduate, Morgan State University, Spring 2026.
For whom	The Morgan State Food Resource Center: paid staff, student volunteers, and the eligible students who use the pantry.
What it is	A web application that runs the pantry's day-to-day operations and captures every event as research-grade data.
Where it impacts	On-campus, in the FRC room. Long-term, any HBCU or community pantry that needs the same instrument.
When it is used	During pantry hours for check-in/check-out, between hours for inventory and reports, year-round for donor and request tracking.
Why	Because primary research data was being generated daily and lost daily. The pantry deserved an observatory, not a clipboard.
How	TypeScript on both ends, Express 5 backend, React 19 frontend, SQLite storage via Drizzle ORM, single binary, single port.
What it took	One developer, one semester, roughly 200 commits, 18 tables, 10 enums, 7 frontend surfaces, ~50 REST endpoints.
What constraints applied	Zero recurring cost, offline-first, FERPA-aware, no SaaS lock-in.
How long	Designed and shipped over a single semester (Spring 2026), with daily iteration during volunteer shifts.

4. System Architecture

The system is a three-tier monolith. Three boxes, one wire each.

```
+----------------------+        HTTP/JSON         +----------------------+
|                      |   <------------------>   |                      |
```

```
|   Browser (React)    |                          |  Express 5 (Node)    |
|   Client SPA         |                          |  REST API + Vite SSR |
|                      |                          |                      |
|  Wouter, TanStack    |                          |  tsx runtime,        |
|  Query, Radix UI,    |                          |  Zod validation,     |
|  Tailwind            |                          |  Drizzle ORM         |
+----------------------+                          +----------+-----------+
                                                             |
                                                             | sync calls
                                                             v
                                                  +----------------------+
                                                  |  SQLite (WAL mode)   |
                                                  |  better-sqlite3      |
                                                  |  data/app.db         |
                                                  +----------------------+
```

Why React 19 + Wouter + TanStack Query

React 19 is the current stable version and gives us the new ref-as-prop and use() hook patterns. Wouter is a 2 KB router with a hooks-based API; we do not need React Router's bigger feature set for 14 routes. TanStack Query handles all server state, caching, and invalidation: the volunteer-facing UI feels instant because every list is cached and only the deltas refetch on mutation.

Why Express 5

Express 5 (released 2024) brings native async error handling, which lets the global error middleware catch promise rejections without `next(err)` boilerplate everywhere. The server runs under `tsx` in development for fast TypeScript-to-Node startup, and bundles to a single CommonJS file via `esbuild` for production (`npm run build` then `npm start`).

Why SQLite via better-sqlite3

The pantry has one process writing at a time. SQLite's WAL mode gives us concurrent readers, sub-millisecond writes on a 41-row transaction log, and zero ops overhead. `better-sqlite3` is synchronous, which simplifies the data layer: the storage adapter wraps every sync call in a resolved promise to satisfy the async `IStorage` interface, and we get transactional integrity for free. The database file lives at `data/app.db`. To back up, copy the file. To migrate, write a script. There is no "database server" to babysit.

Why Drizzle ORM

Drizzle lets us write the schema once in TypeScript (`shared/schema.ts`), in Postgres dialect, and infer both insert and select types directly. The same schema generates Zod validators via `drizzle-zod`. The runtime storage layer (`server/sqlite-storage.ts`) maps Drizzle's Postgres column types to SQLite

types: `numeric` becomes `TEXT` (preserving decimal precision), `boolean` becomes `INTEGER 0/1`, arrays become JSON `TEXT`, and `jsonb` becomes JSON `TEXT`. If the pantry ever needs Postgres, we change one line in `db.ts` and remove the type adapters.

Why no cloud

Three reasons. First, cost: free forever beats $20/month every time. Second, control: the data never leaves the building. FERPA exposure is bounded to a physical machine. Third, operational reality: the FRC's network is unreliable, and a cloud-dependent system would be unusable on the bad days.

5. Schema Design

The 18 tables fall into seven functional groups. Each group answers a different research question.

5.1 Identity (1 table)

Table	Role
`users`	Staff, admin, and volunteer accounts. UUID primary key, role enum (`admin`, `staff`, `volunteer`), bcrypt-style password storage.

5.2 Supply (5 tables)

Table	Role
`inventory_items`	The SKU master. 27 columns covering name, brand, category, barcode, package type, weight (in grams and lbs), value (in cents and USD), allergens, expiration, reorder threshold, and full data provenance (which barcode API matched, with what confidence).
`pack_components`	Components inside a multi-pack or variety-pack SKU. Lets a "case of 12 ramen" decompose into 12 individual units at distribution.
`price_history`	Every cost change writes a row. Source-stamped. Lets us answer "what did this item cost six months ago."
`weight_history`	Every weight change writes a row. Source-stamped, with `is_estimated` flag. Lets us reconstruct measurement quality over time.
`donors`	Donor master. Name, organization, contact, status. Joined to transactions by `transactions.donor` (text key for now; FK migration is straightforward).

5.3 Operations (2 tables)

Table	Role
`settings`	Key/value config (pantry name, pickup window length, reorder thresholds). One row per key, JSON-encoded value.
`migrations`	SQLite-internal table for schema versioning.

5.4 Demand (3 tables)

Table	Role
`clients`	Eligible students. Identifier (campus ID), contact info, household size, eligibility and certification dates, allergies, status. PantrySoft-aligned for future export.
`household_members`	Dependents on a client's household card. Captures household-size truth without forcing dependents to be full clients.
`transactions`	The event log. Every IN (donation) and OUT (pickup) writes one parent row with timestamp, source, donor or client, GPS lat/long, and accuracy.

5.5 Bundling (2 tables)

Table	Role
`item_groups`	Quick-pick bundles (e.g., "Standard Weekly Box"). PantrySoft "kit" equivalent.
`item_group_items`	Junction table: which inventory items go into which group, at what quantity.

5.6 Request System (4 tables)

Table	Role
`requests`	Student-submitted asks for items. Tracks status, reviewer, pickup deadline, fulfillment timestamp, cancellation timestamp, and the eventual transaction ID once filled.
`request_items`	Line items per request. Tracks requested vs approved vs fulfilled quantity and whether stock is reserved.
`request_audit_log`	Immutable change log. Every approval, denial, partial fill, extension, and no-show writes a row with actor,

	action, previous status, new status, and free-text details.
`transaction_items`	Itemized breakdown of every IN and OUT transaction. Each row carries the item name, quantity, weight per unit, and value per unit at the time of the transaction (snapshot pricing, not retroactive).

5.7 Notification (1 table)

Table	Role
`notifications`	Outbound message queue. Targeted by `recipient_type` and `recipient_id`, typed (status update, deadline reminder, fulfillment ready), and flagged read or unread.

Enumerations (10 total)

Four are formal Postgres enums (materialized as `TEXT CHECK` in SQLite):

12. `user_role`: admin, staff, volunteer
13. `transaction_type`: IN, OUT
14. `package_type`: single, multi_pack, variety_pack, case
15. `request_status`: pending, under_review, approved, partially_approved, ready_for_pickup, completed, denied, expired, no_show, cancelled

Six are convention-enforced literal-string enums (typed in TypeScript, validated in Zod, not enum'd at the SQL layer): `client.status`, `donor.status`, `notification.recipient_type`, `notification.type`, `request_audit_log.action`, and `inventory_item.allergens` (string set).

Together, the schema models the pantry as a research instrument: every supply event, every demand event, and every state change is captured with provenance.

6. The Request State Machine

Requests are the most state-heavy entity in the system. A request is born when a student fills out the public portal at `/portal`. It dies in one of five terminal states. Between birth and death, it can move through ten total states, governed by clear transition rules.

```
                                  +-------------+
                                  |   PENDING   | <-- student submits
                                  +------+------+
                                         |
                                         | staff opens
```

```
                                  v
                          +-------------+
                          | UNDER_REVIEW|
                          +-+-----+----++
                            |     |    |
                approve     |     |    |    deny
                            v     |    v
                +-----------+     |   +--------+
                | APPROVED  |     |   | DENIED |  (terminal)
                +-----+-----+     |   +--------+
                      |           |
                      |           | partial approve
                      |           v
                      |   +---------------------+
                      |   | PARTIALLY_APPROVED  |
                      |   +----------+----------+
                      |              |
                      +------+-------+
                             |
                             | items pulled and reserved
                             v
                      +-------------------+
                      | READY_FOR_PICKUP  |
                      +---+-------+---+---+
                          |       |   |
                fulfill   |       |   | expire deadline
                          v       |   v
                  +-----------+   |   +---------+
                  | COMPLETED |   |   | EXPIRED | (terminal)
                  +-----------+   |   +---------+
                                  |
                        client    |   no-show
                        cancels   |
                                  v
                            +-----------+   +----------+
                            | CANCELLED |   | NO_SHOW  |  (terminal)
                            +-----------+   +----------+
```

Why ten states

A simpler "open / closed / fulfilled" model would have hidden the operational reality. Three states matter for staff workflow:

16. **`partially_approved`** captures the common case where a student requests five items but only three are in stock. The request is honored, but the student knows what was cut and why (`request_items.denial_reason` per line).
17. **`ready_for_pickup`** is the reservation lock. Once items are pulled and shelved, they cannot be given to someone else. The request holds them until the pickup deadline (`pickup_deadline`).

18. **`no_show`** is distinguished from `expired`. If staff explicitly mark a no-show after the pickup window, the audit trail records that the student was contacted and did not appear; if no one ever marked it, it auto-expires. This distinction matters for follow-up outreach.

Every transition writes a row to `request_audit_log` with the actor, the action verb, the previous status, and the new status. Nothing is silent.

7. Transaction Schema as Primary Data

The pantry's research value lives in the `transactions` table. Each transaction is a research observation.

```
// transactions table (server/sqlite-storage.ts shape)
{
  id: uuid,
  type: "IN" | "OUT",            // donation vs pickup
  timestamp: ISO 8601 string,    // when the event happened
  source: text | null,            // for IN: drive name, store, etc.
  donor: text | null,             // for IN: donor name (joins to donors)
  clientId: uuid | null,          // for OUT: which client picked up
  clientName: text | null,
  latitude: double | null,        // GPS at time of transaction
  longitude: double | null,
  accuracy: double | null,        // GPS accuracy in meters
  createdAt: ISO 8601 string
}
```

Each transaction has 1 to N child rows in `transaction_items`:

```
{
  id: uuid,
  transactionId: uuid,
  inventoryItemId: uuid,
  name: text,                       // snapshot at time of transaction
  quantity: int,
  weightPerUnitLbs: numeric(10,4),  // snapshot
  valuePerUnitUsd: numeric(10,2)    // snapshot
}
```

Four properties make this research-grade:

19. **GPS-stamped.** Every check-in and check-out captures the device's lat/long and accuracy. We can verify the transaction happened in the FRC room (and not, say, in someone's car) without trusting any human.
20. **Weighed.** Every line carries weight per unit. Total transaction weight is `SUM(quantity * weight_per_unit_lbs)`. The 41 demo transactions sum to 791.8 lbs. This is reportable to grant funders without manual recounting.

21. **Valued.** Every line carries value per unit at transaction time. This is a snapshot, not a foreign key, so retroactive price changes do not corrupt historical totals. Total demo value is computable directly from the table.
22. **Itemized.** No "miscellaneous food" rows. Every line is tied to an `inventory_items.id`, which means every transaction is decomposable into the exact SKUs that moved. This is what lets the dashboard answer "which items went where."

The combination is the point. A donation that is GPS-stamped, weighed, valued, and itemized at intake produces an evidence record stronger than what most municipal food banks generate. The same record powers the operations dashboard and the research export.

8. Frontend Surfaces

The client SPA (`client/src/App.tsx`) routes to seven operational surfaces and two public surfaces. All nine surfaces read and write the same SQLite database through the same REST API. The user experience is tailored per role; the data model is unified.

Route	Surface	Audience	Purpose
/	Dashboard	Staff	Real-time stats: weight on hand, value on hand, recent transactions, low-stock items, request backlog.
/inventory	Inventory	Staff, Volunteer	SKU management. Add, edit, scan barcode (lookup against UPC Item DB, Open Food Facts, USDA FoodData Central in parallel; pick the highest-confidence result).
/clients and /clients/:id	Clients	Staff	Client master with PantrySoft-aligned fields. Household members, allergies, eligibility dates.
/donors and /donors/:id	Donors	Staff	Donor master. Per-donor history of donations, total weight delivered, total value.
/check-in	Check-In	Volunteer	Tap-fast donation intake. Pick donor, scan items, capture weight, GPS-

			stamp, save.
`/check-out`	Check-Out	Volunteer	Tap-fast pickup. Pick client, scan items or pull a quick-pick bundle, GPS-stamp, save.
`/requests`	Requests	Staff	Triage the request queue. Approve, deny, partial-approve, mark ready, fulfill, cancel, mark no-show, extend deadline, add note.
`/reports`	Reports	Staff	Aggregate views: weight by category, donor leaderboard, demand by item, value moved by month.
`/activity`	Activity	Staff	Audit-log viewer. Every request status change, every inventory edit, scrolling timeline.
`/settings`	Settings	Admin	Pantry-wide config (name, pickup window length, default thresholds).
`/portal`	Student Portal	Student (public)	Submit a request. Pick items from the public-safe inventory view, give campus ID and contact, submit.
`/kiosk`	Kiosk	Walk-up student	Touch-optimized self-service for in-room pickup. Looks up an existing approved request by identifier, confirms identity, hands off to volunteer for fulfillment.

Seven surfaces over one database, two public surfaces over a sanitized read-only view (`/api/public/inventory`). No schema duplication. No data sync.

9. Engineering Constraints

Offline-first

The system runs on `localhost:5000` by default and is bound to `0.0.0.0` so other devices on the LAN (a kiosk tablet, a volunteer's phone) can hit it directly. There is no external service the system requires at runtime. Barcode lookups against external APIs are best-effort: if the network is down, the volunteer types the item name manually and the system functions normally. The database file is local. The frontend is local. The server is local.

Single-writer

SQLite in WAL mode supports many concurrent readers and one writer. For a single-pantry deployment with one or two volunteers at the desk, this is more than enough: writes are sub-millisecond, and the busy-timeout pragma (5 seconds) handles the rare write contention gracefully. Multi-writer is a non-feature for v1.

FERPA-aware

The system stores the minimum personally identifiable information needed to operate: name, campus identifier, contact info, household size, eligibility dates. It does not store SSN, race, ethnicity, income, or any sensitive category. The application binds to `0.0.0.0` only on a controlled LAN and uses CORS allow-listing in production via the `ALLOWED_ORIGINS` environment variable (`server/index.ts` lines 30 to 58). The database file is on disk in a controlled physical environment. There is no cloud egress.

Sub-second UX

TanStack Query caches every list. Mutations invalidate only the affected key. The dashboard and the inventory page render from cache on every navigation; the network request happens in the background and updates the cache when it returns. The volunteer at the desk never sees a spinner on a hot path. The 41-row demo transaction set renders in well under 100 ms on a five-year-old laptop.

No SaaS dependency

`package.json` lists 75 production dependencies. Zero of them are SaaS clients. Every dependency is either a UI library (Radix, Tailwind, Lucide), a data library (Drizzle, Zod, TanStack Query), a runtime (Express, React, better-sqlite3), or a utility (date-fns, clsx, papaparse). The barcode lookup endpoint can call three free public APIs (USDA FoodData Central with `DEMO_KEY`, Open Food Facts, UPC Item DB free tier) and three optional paid ones (Nutritionix, UPC Item DB paid, USDA paid). All paid calls are silently skipped when no API key is set.

10. Trade-offs and Limitations

This system is honest about what it does not do.

23. **No multi-pantry federation yet.** Every deployment is a single pantry. If Morgan State opens a second FRC, or if another campus adopts the system, federating two databases is a Postgres migration and a sync layer that does not exist yet. The schema is portable; the federation logic is not built.
24. **No demographic capture.** The system stores no race, ethnicity, income, gender, or other sensitive attribute. This is a deliberate choice for FERPA reasons and student dignity. It also means we cannot answer questions like "are first-generation students using the pantry at higher rates than legacy students" without a separate, IRB-approved instrument.
25. **No predictive model.** The system records what happened. It does not yet forecast what will happen next week. Demand forecasting, donor gap analysis, and "what will we run out of by Thursday" are real research questions that the data supports, but the model is not built.
26. **Demo data is seeded.** The 41 transactions, 27 SKUs, 10 students, 6 donors, and 791.8 lbs in the current `data/app.db` are seed data, not live operational data. The system has been used in dry runs and internal demos. Live deployment with real student data requires Morgan State IRB review and an FRC staff training session, both pending as of April 2026.
27. **No native mobile app.** The kiosk and student portal are responsive web pages. They work fine on a phone or a tablet, but there is no offline-capable iOS or Android client.
28. **Single point of failure.** One machine, one SQLite file. A backup script exists (`scripts/`), but the recovery story today is "restore from yesterday's copy." A production deployment would add a hot-standby and an off-site backup.

11. Future Work

In rough order of urgency:

29. **IRB approval** (Morgan State Institutional Review Board): required before live student data is collected for research purposes. Operational use does not need IRB; research publication does.
30. **Postgres federation:** swap `better-sqlite3` for pg driver, run migrations, add a `pantry_id` column to every fact table, deploy a multi-tenant instance for a hypothetical HBCU pantry consortium.
31. **Demand forecasting:** train a simple time-series model on `transaction_items` to predict per-category outflow for the next 7 days. Surface the prediction on the dashboard alongside reorder thresholds.
32. **Donor gap analysis:** join donor inflow against client outflow by category. Surface "donor X reliably fills the gap for category Y" and "category Z is structurally undersupplied." This is the highest-value report for development staff.

33. **Public dashboard:** a read-only, anonymized weekly report (total weight moved, total value, top categories, no individual identifiers) suitable for the Morgan State website and grant applications.
34. **MIT-licensed public release:** the system is already MIT-licensed in this repository. Public release means writing a deployment guide, recording a setup video, and offering it to other HBCU food pantries. (Companion case study at https://github.com/PadiusD1/morgan-frc-research-paper.)

12. Reproducibility

The system runs on any machine with Node.js 20 or newer. There is no database server to install. There is no cloud account to provision.

```
git clone https://github.com/PadiusD1/ClaudeMSUfoodResourceCenter.git
cd ClaudeMSUfoodResourceCenter
npm install
npm run dev
# open http://localhost:5000
```

That is the full setup. The first run creates `data/app.db` automatically and runs all CREATE TABLE migrations. The dev server is `tsx server/index.ts` (via `npm run dev`), which serves both the API and the Vite-powered React client on a single port. For production, `npm run build` bundles to `dist/index.cjs` and `npm start` runs it.

Optional environment variables (all are optional, all live in `.env`):

Variable	Purpose
`PORT`	Server port (default 5000).
`ALLOWED_ORIGINS`	Comma-separated allow-list for production CORS.
`UPCITEMDB_API_KEY`	Higher rate limits on UPC Item DB.
`NUTRITIONIX_APP_ID`, `NUTRITIONIX_APP_KEY`	Nutritionix barcode lookup.
`USDA_API_KEY`	USDA FoodData Central (defaults to DEMO_KEY).

The pantry runs without any of them.

A pantry is not a charity program. A pantry is an observatory.